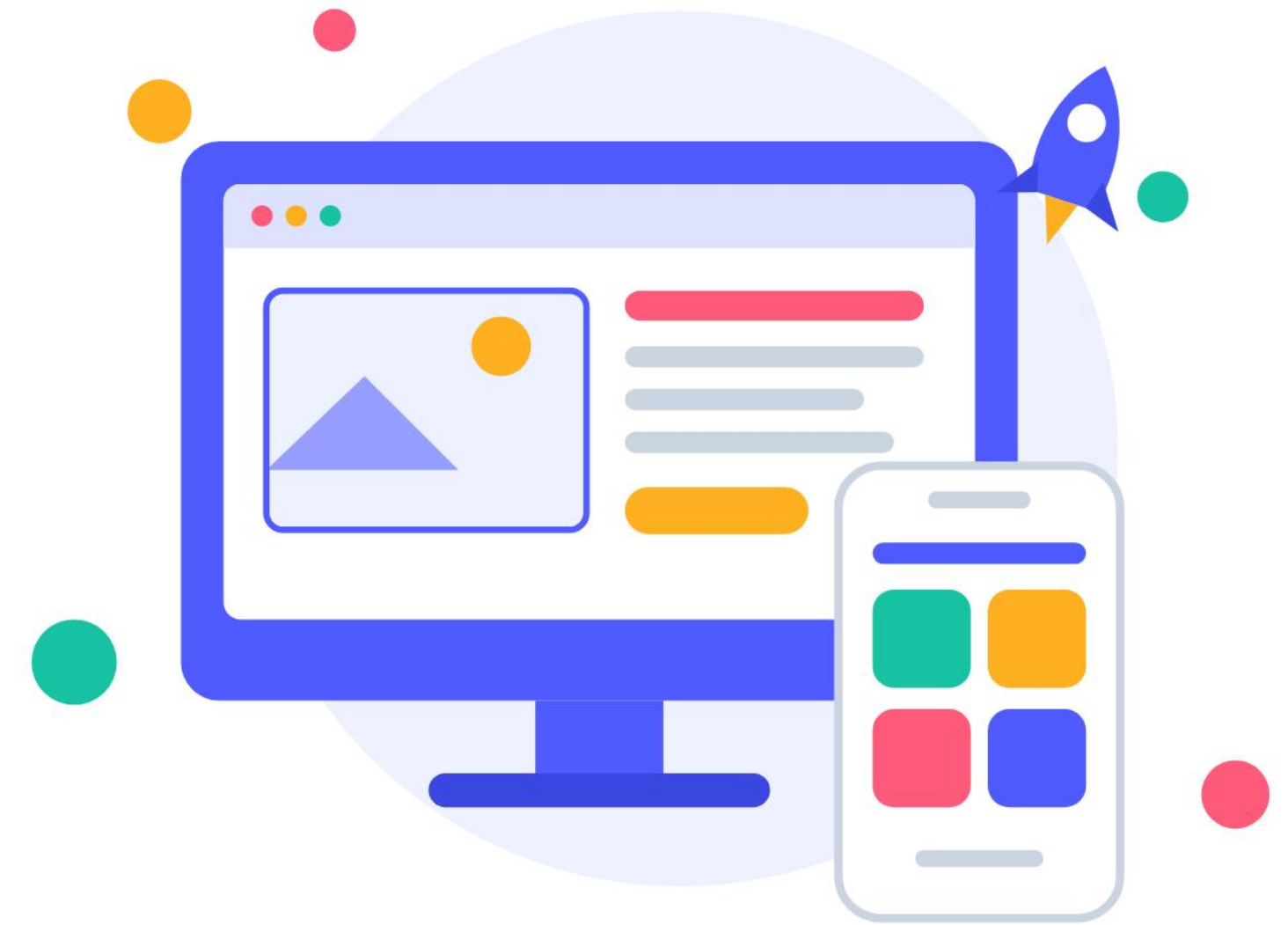


WEEK 04 · FLOW · POSITION · FLEXBOX

WHERE THE BOXES GO

Last week you learned to style a box. Today you learn to put one somewhere.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



WHERE WE ARE

Week 4 of 12 (week 13 is the exam). You have a page with a design language and no layout.

THE NEXT 180 MINUTES

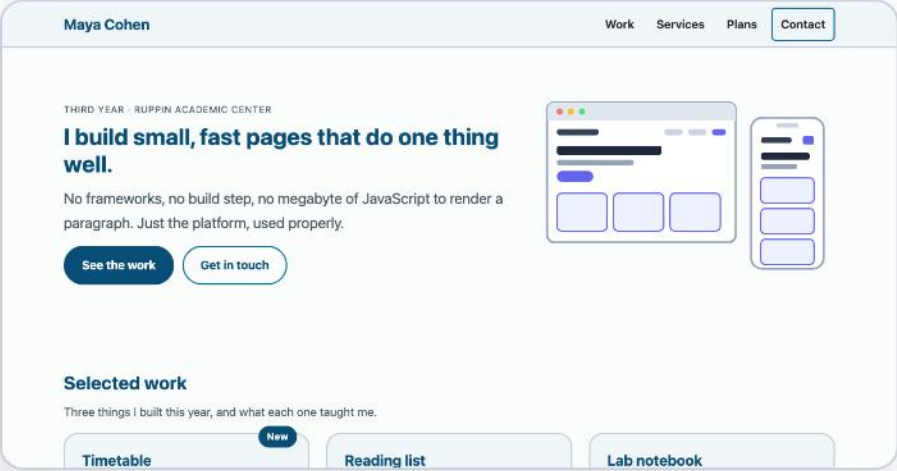
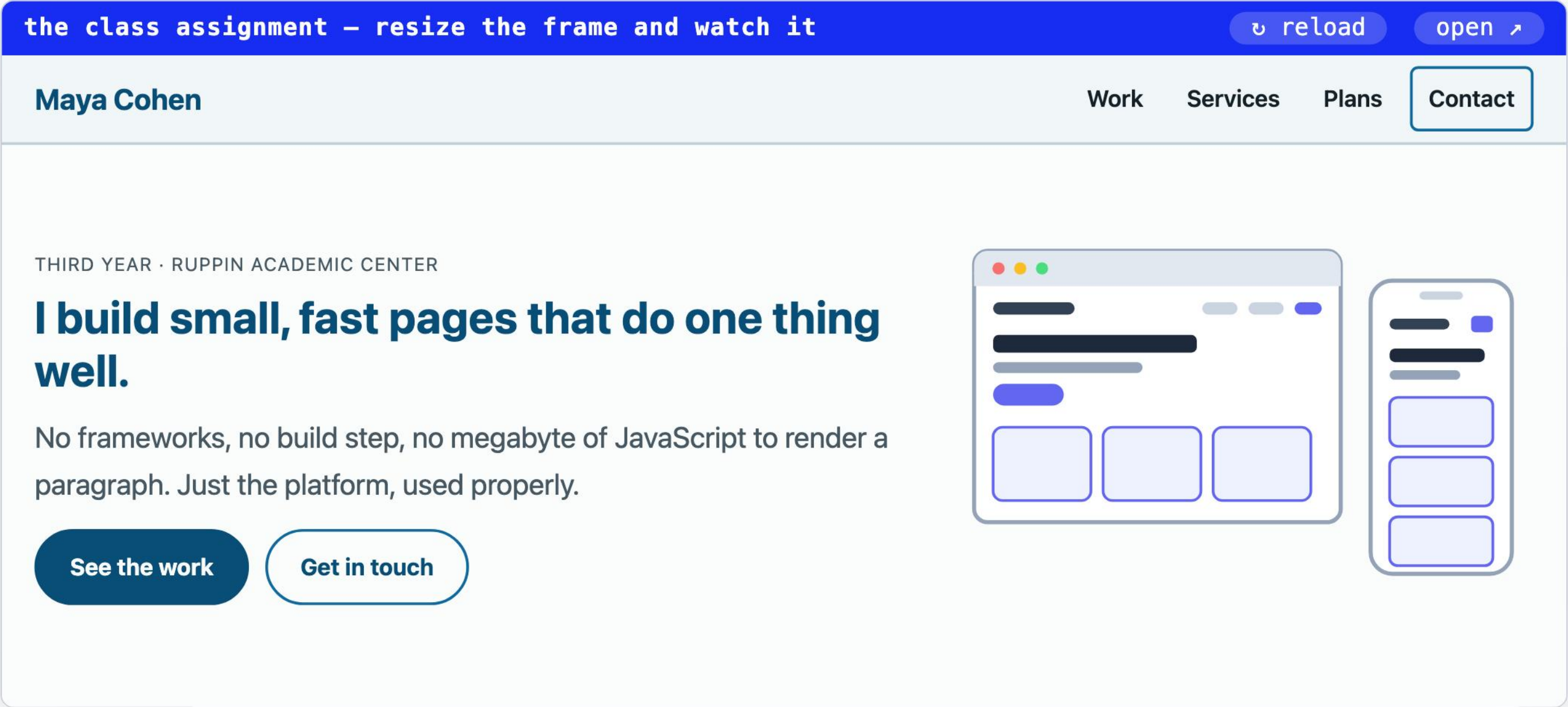
Five cycles, you typing in every one: a navbar · a badge and a sticky header · a card row that wraps · one prompt to Claude · a menu that closes on a phone. Break at 90. The assignment at 145.

WHAT THIS UNLOCKS

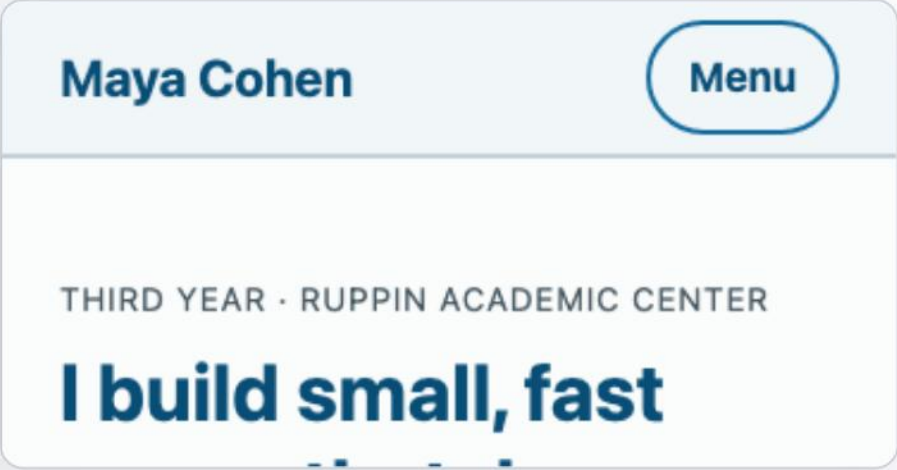
Week 5 is Grid, and the decision rule "Grid for the page, Flex inside the component" only means something once you know what Flex is.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180



targets/week-04.png · 1280



375 · the menu closes (the one query)

► **THE POINT** — everything that moves when you resize this does so without a line that mentions a screen size — except the one media query, only for the menu. You type nearly every piece of it today — the few long blocks are marked **COPY THIS** — once in a cycle, once in the assignment.

AI THIS WEEK

TUTOR. IT EXPLAINS; YOU TYPE.

LAYER · WEEKS 1–5 · PRIVATE TUTOR

ALLOWED

Ask it to explain the two axes, a stacking context, `flex-basis`. Ask it to criticise CSS *you* typed. Ask "why doesn't my rule apply?" — then check the answer in DevTools, not in your gut.

NOT ALLOWED

Pasting generated CSS into the assignment. **You type nearly every piece.** Long blocks are marked *copy this*; the rest is yours. What to check this week: it copies the dialect of the CSS you paste and reaches for old habits. You get the answer that works at the width you looked at — so drag the width slowly from 1600 to 320 and test the one you did not.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one prompt, twice, in whatever chat you have.

CYCLE 1 · BEFORE YOU TYPE

NOTHING YOU HAVE WRITTEN SO FAR HAS PLACED ANYTHING

Every page you have made was laid out by a rule you were never told. It is **two sentences long**.

A BLOCK BOX

Takes the full width offered; the next one starts underneath.

p h1 div section li

AN INLINE BOX

Takes the width of its content; the next one continues on the line.

a span strong em img

LOOK AT THE BOXES THE BROWSER MADE

demos/01-normal-flow – no layout CSS at all

↺ reloadopen ↗

☒ outline every box☐ show the line boxes☐ show the containing block

■ block box

■ inline box

■ line box

■ containing block

Normal flow

This paragraph is a **block box**. It takes the whole width it is offered, and the next block box starts underneath it. Nobody asked for that: it is what the browser does when you say **nothing**.

Inline content flows

Inside a block box, text and **inline boxes** are poured into **line boxes** — horizontal strips as wide as the block, filled one after another until the words run out. An **inline box** is exactly as wide as its content, and an  sits on the text baseline like a very large letter.

- A list item is a block box too.
- So they stack.

Everything on this page was laid out before you wrote a single layout rule.

Hover an element in the page above.

Hover any element in the page above to measure it. Nothing here is drawn by hand — the strips and the frame are positioned from `getClientRects()`.

▶ **HANDS-ON** demos/01-normal-flow/ — Turn on the outlines. Blue is a block box, orange is an inline box. Then hover anything and read what it was offered.

5

← TAUGHT · 4

CYCLE 1 · BEFORE YOU TYPE

display DECIDES WHAT KIND OF BOX

It is not decoration and it is not visibility. It answers one question: *what sort of box does this element generate?*

Last week a **width** on a link did nothing: a link makes an inline box.

block

Full width offered · stacks · width, height and all margins work.

inline

Content width · flows in a line · width, height and vertical margin ignored.

inline-block

Content flow, block sizing. Both. The famous problem: whitespace between two inline-blocks renders as a gap.

none

No box at all. Not "invisible" — absent.

display

— WHAT KIND OF BOX

VALUE	LINE BEHAVIOUR	WIDTH / HEIGHT	VERTICAL MARGIN
block	starts a new line, takes the width offered	yes	yes
inline	flows in the line, content width	ignored	ignored
inline-block	flows in the line	yes	yes
flex	block-level container, children become flex items	yes	yes
inline-flex	inline-level container, children still flex items	yes	yes
none	no box at all	—	—

`inline-block` is an *inline* box, so the whitespace between two of them in the source is rendered as a space. That is the gap.

THE SAME THREE DECLARATIONS, THREE TIMES

demos/02-display/01-inline-wont-size

↺ reload

open ↗

THE SAME DECLARATIONS, THREE TIMES

display: inlineText before **the link** and text after it, on the same line.**display: inline-block**Text before **the link** and text after it, on the same line.**display: block**

Text before

the link

► **HANDS-ON** demos/02-display/01-inline-wont-size/ — Guess the first link's height before you read the measured number.

FOUR WAYS TO HIDE, AND WHAT EACH ONE COSTS

METHOD	KEEPS ITS SPACE	REACHABLE BY TAB	READ BY A SCREEN READER
<code>display: none</code>	no	no	no
<code>hidden</code> attribute	no	no	no
<code>visibility: hidden</code>	yes	no	no
<code>opacity: 0</code>	yes	yes	yes

`opacity: 0` is **only transparent**. It is still there, still focusable and still clickable — which is a bug that is very hard to see.

FOUR WAYS TO MAKE SOMETHING DISAPPEAR

`display: none`

No box. No space, no tab stop, not read aloud. The honest "gone".

`visibility: hidden`

Invisible, **keeps its space**. Not focusable, not read aloud.

`opacity: 0`

Only transparent. Still takes space, still focusable, still read aloud, **still clickable**.

► **HANDS-ON** `demos/02-display/03-none-vs-hidden/` — Choose `opacity: 0`, then click where the button should be and watch the counter.

CYCLE 1 · BEFORE YOU TYPE

A FLEX CONTAINER, AND ITS ITEMS

flex container The element you write `display: flex` on. It lays its children out side by side, in a row.

flex item Each **direct child** of the container. A grandchild is not an item.

gap Space *between* the items. Never outside them.

flex-wrap A row never wraps by itself: it squeezes. `wrap` lets items drop to a new line.

```
1 .links {  
2   display: flex;    /* the container */  
3   gap: 8px;        /* between items */  
4 }
```

flex container



What the browser draws: one row, equal gaps.

FLEXBOX · ON THE CONTAINER

display	<code>flex</code> · <code>inline-flex</code>
flex-direction	<code>row</code> · <code>row-reverse</code> · <code>column</code> · <code>column-reverse</code>
flex-wrap	<code>nowrap</code> · <code>wrap</code> · <code>wrap-reverse</code>
justify-content	<code>flex-start</code> · <code>flex-end</code> · <code>center</code> · <code>space-between</code> · <code>space-around</code> · <code>space-evenly</code> — main axis
align-items	<code>stretch</code> · <code>flex-start</code> · <code>flex-end</code> · <code>center</code> · <code>baseline</code> — cross axis
align-content	Same values, but for the <i>lines</i> . Only does anything with <code>wrap</code> .
gap	<code>row-gap</code> and <code>column-gap</code> . Space <i>between</i> items only, never outside.

THE SAME NAVBAR, FOUR WAYS

demos/06-flex/01-why-flex

normal flow

inline-block

float

flex

THE MARKUP NEVER CHANGES. ONLY THE STYLESHEET DOES.

Ruppin FE

Home

Work

About

Sign in

This paragraph comes after the bar in the markup.

/* nothing at all */

bar height187px

links on one rowno – they are on 4 rows

"Sign in" at the end yes

Every is a block box, so they stack. This is not a bug – it is the default, and it is the right default for a list.

The question the rest of act 5 answers: how do you ask for a row?

▶ **HANDS-ON** demos/06-flex/01-why-flex/ — Run the float version and work out why the gold bar has no height.

CYCLE 1 · BEFORE YOU TYPE

TWO AXES, AND THEY MOVE

flex-direction `row` (the default) or `column`: which way the items are laid out.

main axis The direction items are laid out. Set by `flex-direction`.

cross axis Perpendicular to it. Always.

justify-content Distributes along the **main** axis:
`flex-start` · `flex-end` · `center` · `space-between`.

align-items Aligns along the **cross** axis: `stretch` (the default) · `flex-start` · `flex-end` · `center`.

Where the axes point

FLEX-DIRECTION	MAIN	CROSS
<code>row</code>	horizontal →	vertical ↓
<code>row-reverse</code>	horizontal ←	vertical ↓
<code>column</code>	vertical ↓	horizontal →
<code>column-reverse</code>	vertical ↑	horizontal →

Neither is "horizontal": which is main and which is cross was decided one line earlier.

MISCONCEPTION 3 · WATCH THE AXIS MOVE

demos/06-flex/playground – every property, live

↺ reload

open ↗

THE CONTAINER

flex-direction

row

flex-wrap

nowrap

justify-content – horizontal now

flex-start

align-items – vertical now

stretch

align-content (needs wrap)

normal

gap

10px

THE SELECTED ITEM

flex-grow

0

flex-shrink

main axis →

cross axis ↓

1

2

3 – a taller item, with more words in it

4

5

```
.container {
  display: flex;
  gap: 10px;
}
```

container	1278 × 240	main axis is horizontal
item 1	56 × 216	← selected
item 2	56 × 216	
item 3	190 × 216	
item 4	56 × 216	
item 5	56 × 216	

▶ **HANDS-ON** [demos/06-flex/playground/](#) — Switch to `column`. The arrows turn, and the labels rewrite themselves.

CYCLE 1 · 29 MIN

FROM A STACK TO A NAVBAR

A link, then a list with bullets — exactly where normal flow put them. A handful of declarations later it is a navbar, and not one of them is a width.

- 1 I DO12 MIN

5 on the slides above, 7 in the editor · live-code/01-stack-to-row · from an empty bar to brand-left, links-right
- 2 WE DO6 MIN

you type it on your copy · compare with checkpoint 1
- 3 YOU DO7 MIN

warmup/ — the gold bar, five declarations, and the push that needs two
- 4 DEBRIEF4 MIN

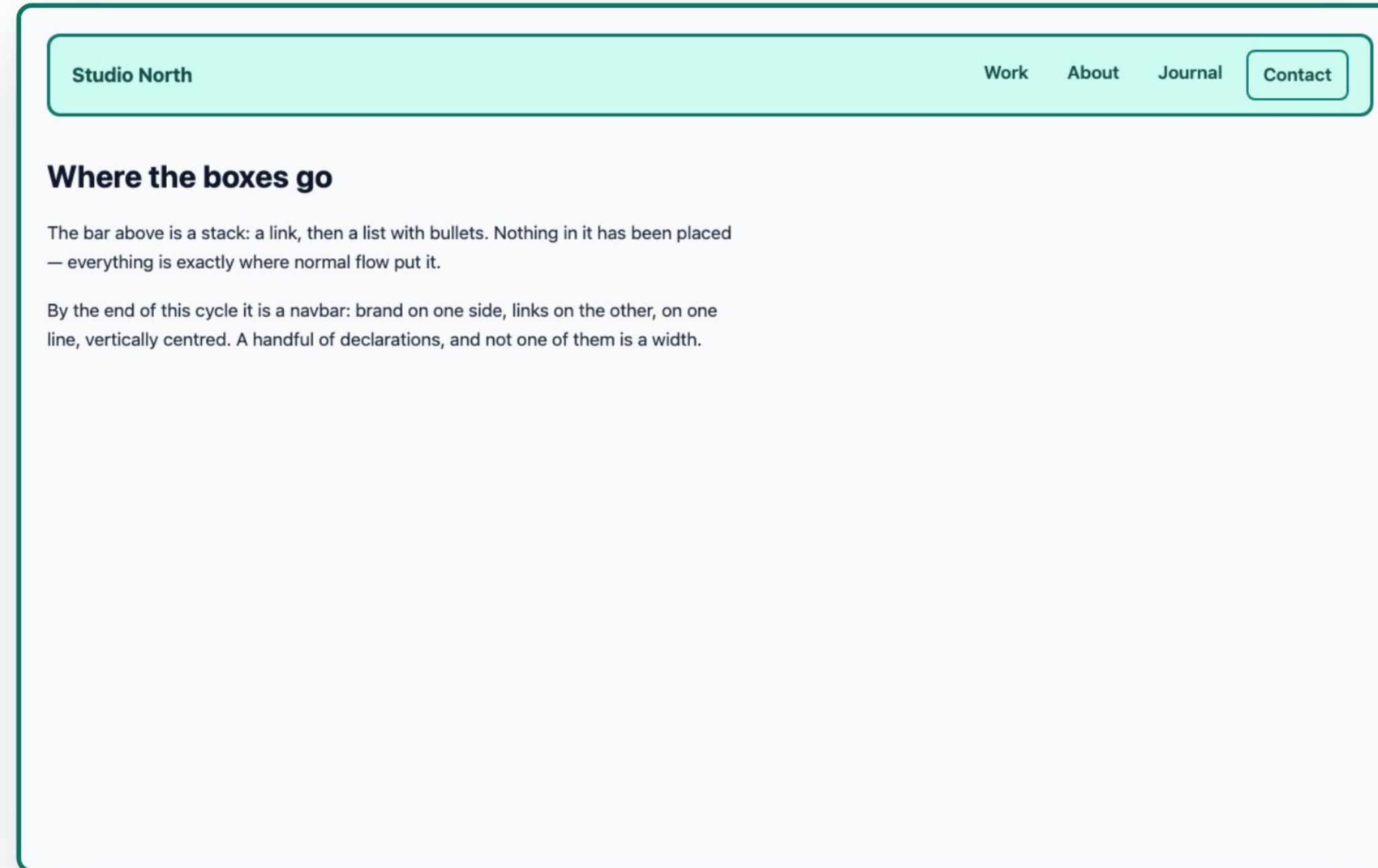
flex on the item · the ul padding · the auto margin, named · ↗ deeper

YOU DO · AI-FREE



CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/01-stack-to-row/checkpoint.png` is this picture at 1280; `live-code/01-stack-to-row/end/` is the code behind it. Brand left, links pushed right, one line, Contact outlined.

CYCLE 1 · AFTER YOU TYPED IT

AN AUTO MARGIN ABSORBS THE FREE SPACE

THE HABIT IT IS NOT

Reaching for `margin: 0 auto`, then a negative margin, then another one, until it looks right at the width you happen to be at.

THE ONE REAL USE, IN FLEX

An auto margin **absorbs the free space**. In a flex row that makes it a positioning tool:
`margin-inline-start: auto` pushes one item, and everything after it, to the end.

AND THE OTHER ONE

`margin-block-start: auto` in a flex *column* pins an item to the bottom. That is how pricing buttons line up.

CYCLE 2 · BEFORE YOU TYPE

EVERY BOX IS MEASURED AGAINST ANOTHER BOX

containing block The box your box's sizes and offsets are resolved against.

in normal flow Your nearest block-level ancestor's **content box**.

why you care `width: 50%` is 50% *of that*. So is `left: 10%`.

Change which box that is, and every percentage in the element changes meaning without you touching it. **Cycle 2 is entirely about changing it.**

```
1 .parent {  
2   width: 600px;    /* the containing block */  
3 }  
4  
5 .child {  
6   width: 50%;      /* 300px */  
7 }
```

WHAT EACH BOX IS MEASURED AGAINST

- in normal flow** The nearest block-level ancestor's **content box**.
- absolute** The nearest ancestor with `position` other than `static` — its **padding box**.
- fixed** The viewport — *unless* an ancestor has a `transform`, `filter` or `perspective`, in which case that ancestor.
- sticky** Its nearest **scroll container**. No scroll container, no sticking.

Every percentage in an element — `width: 50%`, `left: 10%`, `padding: 5%` — resolves against this box.

Percentage heights are the exception that bites: they need the containing block to have a *definite* height. If it is `auto`, `height: 100%` resolves to `auto` too.

Use `min-height: 100dvh` instead. It asks nobody.

MISCONCEPTION 5 ·

height: 100%

OF WHAT?

demos/05-overflow/02-height-100

↺ reload

open ↗

☐ height:100% on <html>

☐ height:100% on <body>

☐ height:100% on .wrapper

instead: min-height on the box itself

THE BLUE BOX SAYS height: 100%. IT ALWAYS HAS.

html

body

.wrapper

height: 100%

.target { height: 100% } ← never changes in this demo

the chain above it:

htmlheight: auto0px ← auto: as tall as its children

bodyheight: auto0px ← auto: as tall as its children

.wrapperheight: auto0px ← auto: as tall as its children

▶ **HANDS-ON** `demos/05-overflow/02-height-100/` — Turn the three switches on left to right. Nothing happens until the last one.

ASK NOBODY: USE A VIEWPORT UNIT

NEEDS THREE ANCESTORS TO COOPERATE

```
1 html, body { height: 100%; }  
2 .wrapper { height: 100%; }  
3 .hero { height: 100%; }
```

NEEDS NOBODY

```
1 .hero {  
2   min-height: 100dvh;  
3 }
```

`dvh` is the *dynamic* viewport height — the one that stays correct when a phone browser slides its toolbar away.

CYCLE 2 · BEFORE YOU TYPE

TAKING A BOX OUT OF THE CONVERSATION

Normal flow is a negotiation between siblings. `position` is how one box *leaves* it — and the cost is that nothing reserves its space any more.

static

The default. In flow. Offsets ignored.

relative

In flow, keeps its space, moves visually.

Anchors its children.

absolute

Out of flow. Measured from the nearest positioned ancestor (*positioned* = any `position` except `static`).

fixed

Out of flow. Measured from the viewport. Does not scroll.

sticky

In flow, then pins at a threshold. Needs an offset such as `top: 0`, or it does nothing.

offsets

`top` `right` `bottom` `left`, or `inset-block-start` and its siblings: how far from the anchor.

position

— THE FIVE VALUES

VALUE	IN NORMAL FLOW	OFFSETS MEASURED FROM	ON SCROLL
static	yes	offsets ignored	scrolls
relative	yes — keeps its space	its own original position	scrolls
absolute	no	nearest positioned ancestor	scrolls
fixed	no	the viewport	stays put
sticky	yes — keeps its space	itself, then its scroll container	scrolls, then pins

"Out of flow" means nothing reserves its space any more: the siblings close the gap, and a parent with no other content collapses to nothing.

CYCLE 2 · BEFORE YOU TYPE

WHEN THE CONTENT DOES NOT FIT

A box has a size; its content can be bigger. `overflow` says what the box does with the extra. A box that scrolls its own content is a **scroll container**.

visible

The default. Content paints outside the box, over whatever is under it.

hidden

Clipped — and the box becomes a scroll container. The content is still findable by `Ctrl-F` and by `Tab`.

auto

A scrollbar only when one is needed. The one you usually want.

► **HANDS-ON** [demos/05-overflow/01-overflow-values/](#) — Pick `hidden`, then press `Ctrl-F` in the frame and search for **BURIED**.

overflow

VALUE	CLIPPED	SCROLL CONTAINER	CONTENT STILL REACHABLE
visible	no — paints outside	no	yes
hidden	yes	yes	yes — Tab and Ctrl-F scroll to it
scroll	yes	yes, bars always shown	yes
auto	yes	yes, bars when needed	yes
clip	yes	no	no

Any scroll container becomes the box a `sticky` descendant sticks inside — which is the usual reason a correct `sticky` does nothing.

ALL FIVE, ON ONE SCROLLING PAGE

demos/03-position/02-five-values

↺ reload

open ↗

static

relative

absolute

fixed

sticky

THE DASHED FRAME IS THE CONTAINER. IT IS `position: relative`.

Text above the box.

where it would be in normal flow

the box

```
position: static
```

still in normal flow?	yes – this IS normal flow
offsets measured from	nothing. top and left are ignored entirely.
when you scroll	scrolls with the page
container height now	0px
computed position	static

The default. Every element on every page you have written so far.
Note that top/left do nothing here – offsets need a position first.

► **HANDS-ON** `demos/03-position/02-five-values/` — Start on `relative` and point at the dashed ghost: **THAT SPACE IS STILL RESERVED.**

A SCROLL CONTAINER IS ALSO WHAT **sticky** STICKS INSIDE

The most common reason a perfectly correct `position: sticky` does nothing is **an overflow somebody added to an ancestor for an unrelated reason**.

A sticky element can only travel inside its own scroll container. If that container is no taller than the element, it never has anywhere to stick.

HOW TO FIND IT

Walk up the ancestors in DevTools looking for `overflow` that is not `visible`. It is almost always three or four levels up.

CYCLE 2 · BEFORE YOU TYPE

absolute — MEASURED FROM WHAT, EXACTLY?

- 1 Walk up**
from the element, parent by parent.
- 2 Stop**
at the first *positioned* ancestor: one whose `position` is not `static`.
- 3 Measure**
the offsets from that ancestor's padding box.
- 4 If there is none**
, fall back to the page itself.

Step 4 is where everybody gets hurt: it **always succeeds**, and just anchors somewhere you did not mean.

Ask the browser which box it chose: `el.offsetParent` in the console. **Copy this** — JavaScript, week 8.

```
1 .card {  
2   position: relative; /* ← the anchor */  
3 }  
4  
5 .badge {  
6   position: absolute;  
7   top: -10px;  
8   right: -10px;  
9 }
```

MISCONCEPTION 1 · THE BADGE THAT FLEW AWAY

[demos/03-position/01-absolute-adrift](#)[↶ reload](#)[open ↗](#)☐ **position: relative** on the middle card ☐ show the ancestor chain

THE BADGE BELONGS TO THE MIDDLE CARD

the page

NEW

one

two

three

The badge landed 525px away from the card it belongs to.

```
.badge { position: absolute; top: -10px; right: -10px; }
```

– unchanged. The badge never moves in this demo.

walking up from the badge, looking for a positioned ancestor:

```
<div#card>          static
<div#row>           static
<div.page-root>     relative ← this one anchors the badge
<body>             static
<html>             static
```

The walk went straight past the card – it is `static`, so it does not count – and stopped at the page. The offsets are measured from THERE.

"If it aligns to the page instead of the box you expected, you probably forgot `position: relative` on the ancestor."

▶ **HANDS-ON** [demos/03-position/01-absolute-adrift/](#) — The badge's CSS never changes. Turn on the ancestor chain.

CYCLE 2 · BEFORE YOU TYPE

z-index, THE MODEL EVERYONE HAS

Bigger number in front. Default is `auto`. It orders positioned boxes (and flex items).

All three of those are true, and together they are still not enough to explain the example on the right.

```
1 .dropdown {  
2   position: absolute;  
3   z-index: 999;  
4 }  
5  
6 .header {  
7   position: sticky;  
8   z-index: 10;  
9 }
```

999 > 10. So the dropdown is in front. Obviously.

CYCLE 2 · 27 MIN

THE BADGE, THE ANCHOR, AND THE HEADER THAT STAYS

Three uses of `position`, and not one of them places a column: a badge on a corner, the box it is measured from, a header that stays — above the content.

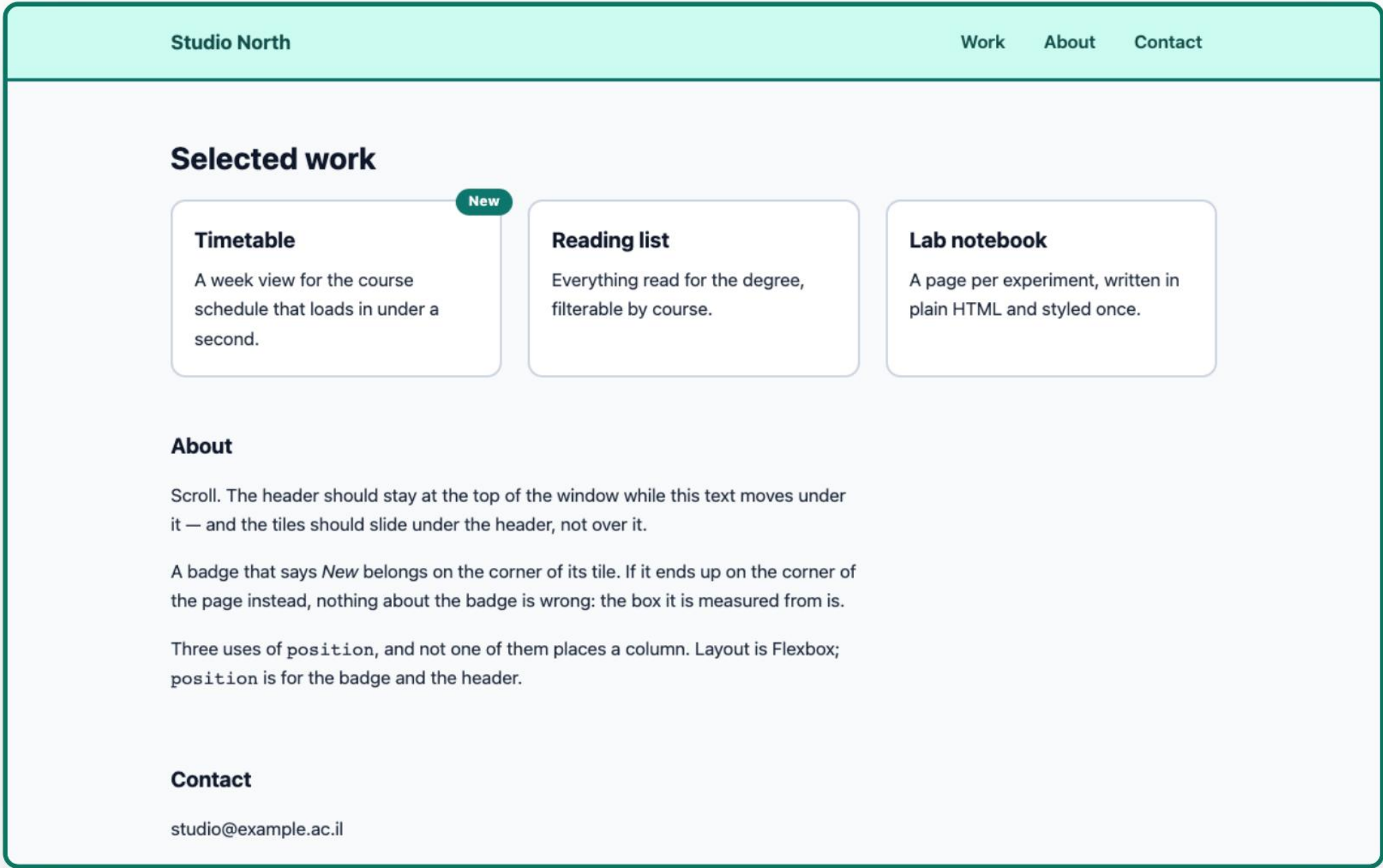
1 I DO13 MIN 6 on the slides above, 7 in the editor · live-code/02-badge-and-anchor · the badge flies, the anchor, sticky, the 999 that loses	2 WE DO5 MIN type it · checkpoint 2 · <code>offsetParent</code> in the console (copy this — JavaScript, week 8)	3 YOU DO6 MIN a second badge, bottom-left of the third tile · verify with <code>offsetParent</code>	4 DEBRIEF3 MIN the sealed box · the diagnostic sentence · ↗ deeper
---	--	---	--

YOU DO · AI AS TUTOR



CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/02-badge-and-anchor/checkpoint.png` is this picture at 1280; `live-code/02-badge-and-anchor/end/` is the code behind it. The badge on the corner of the first tile, not of the page; the header stays when you scroll and the tiles slide under it.

CYCLE 2 · DEBRIEF

A STACKING CONTEXT IS A SEALED BOX

When an element creates one, its children's `z-index` values are only ever compared **with each other**. Against anything outside, what competes is *the parent's own* `z-index`.

So `999` was never being compared with `10`. It was being compared with its siblings, inside a box whose own `z-index` is `auto` — and `auto` loses to `10`.

Raising the number cannot help. It is the wrong argument. Find the ancestor that made the context; remove what made it one, or give it a `z-index`.

position

`fixed` or `sticky`: always.
`relative` or `absolute`: only with a `z-index` other than `auto`

opacity

anything below `1`

transform

anything other than `none`
(week 5)

filter

anything other than `none`
(week 5)

will-change

naming any of the above

isolation

`isolate` — the only one that is deliberate

MISCONCEPTION 2 · WHEN 999 LOSES TO 10

demos/04-stacking/01-z-index-lies

↺ reloadopen ↗

dropdown z-index 999

☐ opacity: 0.99 on the panel

☐ transform: translateZ(0) on the panel

☐ filter: saturate(1) on the panel

☐ will-change: transform on the panel

☐ isolation: isolate on the panel

THE GREEN DROPDOWN HAS Z-INDEX 999. THE PINK HEADER HAS Z-INDEX 1.

Menu

I am z-index: 999

The dropdown is on top, as asked.

```
.panel    { }                ← creates no stacking context
.dropdown { z-index: 999 }
.header   { z-index: 1 }
```

Both are in the same stacking context – the root – so the two numbers are genuinely comparable, and the bigger one wins.

Now tick any box above. None of them mentions layering.

▶ **HANDS-ON** demos/04-stacking/01-z-index-lies/ — Predict what `opacity: 0.99` will do. Then tick it. Then try 99999.

KEEP THIS SENTENCE: A BOX IN THE PAGE CORNER MEANS A MISSING `position: relative`

If it aligns to the page instead of the box you expected, you probably forgot `position: relative` on the ancestor.

COMPANION · THE COLLAPSED PARENT

Absolute elements won't stretch the parent's height
— parents may collapse.

COMPANION · THE TRANSFORMED ANCESTOR

`fixed` inside a transformed ancestor behaves like `absolute`. Avoid transforming high-level wrappers.

CYCLE 3 · BEFORE YOU TYPE

FLEXBOX: ONE DIMENSION, DONE PROPERLY

Flexbox is designed for components — navbars, cards, toolbars, forms — rather than full page grids.

Flexbox lays items out in a **row or a **column**, with real control over how the space between and around the items is distributed. That is the entire scope, and it is why it is easy.**



CYCLE 3 · BEFORE YOU TYPE

CONTAINER PROPERTIES · ITEM PROPERTIES

ON THE CONTAINER

`flex-direction · flex-wrap ·``justify-content · align-items · gap`

Start with `flex-basis` to set the natural size, then tweak `flex-grow` and `flex-shrink`.

`flex-basis`

The size an item starts from, along the main axis. Default `auto` = its content.

`flex-grow`

Its share of the *leftover* space. Default `0`.

`flex-shrink`

Its share of the *overflow* it gives up. Default `1`.

`flex`

The shorthand: grow, shrink, basis.

`flex: 1 1 17rem` = may grow, may shrink, wants 17rem. `flex: 1` = `1 1 0%`;

`flex: none` = `0 0 auto`.

`min-width`

Default `auto`: an item will not shrink below its content. `min-width: 0` lifts that.

`align-self`

Overrides `align-items` for this one item.

`order`

Lower comes first. **Visual only.**

THE THREE NUMBERS

demos/06-flex/02-grow-shrink-basis

reloadopen

the default (0 1 auto)all three: flex: 1all three: flex: autogrow 2 / 1 / 1a sidebar: 200px + flex: 1

item 1

flex-grow0

flex-shrink1

flex-basisauto

item 2

flex-grow0

flex-shrink1

flex-basisauto

item 3

flex-grow0

flex-shrink1

flex-basisauto

THREE ITEMS IN A 620PX RAIL, WITH TWO 10PX GAPS

one

two

three — the item with a good deal more text in it

container	596px
– two gaps	20px
– Σ basis	403px
free space	173px

► **HANDS-ON** `demos/06-flex/02-grow-shrink-basis/` — The panel does the arithmetic and then prints the browser's measured widths beside it. They match.

THE SHORTHAND YOU WILL ACTUALLY READ

flex: 1 1 1 0% — basis zero, so content is ignored and widths come out equal.

flex: auto 1 1 auto — basis is the content, so wider items stay wider.

flex: none 0 0 auto — never grows, never shrinks.
For the logo.

That first difference is most of the confusion about **flex** in other people's code.

```
1 .card {  
2   /* every card the same width */  
3   flex: 1;  
4 }  
5  
6 .card {  
7   /* wider cards stay wider */  
8   flex: auto;  
9 }
```

THE flex SHORTHAND

YOU WRITE	IT MEANS	USE IT WHEN
<code>flex: 1</code>	<code>1 1 0%</code>	equal widths, content ignored
<code>flex: auto</code>	<code>1 1 auto</code>	wider content stays wider
<code>flex: none</code>	<code>0 0 auto</code>	never grows or shrinks — a logo
<code>flex: 1 1 17rem</code>	as written	the responsive card row
<code>flex: 0 0 200px</code>	as written	a fixed sidebar

The free space is `container - gaps -  $\Sigma$  basis`, and each item gets `basis + (free  $\times$  its grow  $\div$   $\Sigma$  grow)`. When the free space is negative, `flex-shrink` takes over instead.

THE ONE THAT WILL COST YOU AN AFTERNOON

demos/06-flex/03-min-width-auto

reload

open

☐ put a long unbreakable string in the content

☐ min-width: 0 on .content

☐ + text-overflow: ellipsis

flex-shrink on .content

1

A 150PX SIDEBAR AND A flex: 1 CONTENT COLUMN

sidebar
150px

The content column is flex: 1, so it should take what is left.

The row fits its container.

container inner width1498px

the flex row wants1498px

.content min-widthauto

.content flex-shrink1

Nothing is wrong yet. The content is ordinary text, which can wrap, so its minimum content width is small and flex-shrink has room to work.

Tick the first box to give it something that cannot wrap.

▶ **HANDS-ON** demos/06-flex/03-min-width-auto/ — Turn on the long string. Now try to fix it with flex-shrink . Drag it to 10. Nothing.

42

← TAUGHT · 38

order AND row-reverse — AND THE WARNING

Both change the **visual** order of items without touching the HTML. Both are genuinely useful, and both are a trap.

Tab order, screen-reader order and copy-paste order all follow the **DOM**. Reorder visually and you have made three orders that disagree — and only one of them is the one you tested.

Rule: if the order matters, fix the HTML. Use these for presentation, never for meaning.

align-self

Overrides `align-items` for one item.

order

Default 0. Lower comes first. Negative is allowed.

align-content

Only does anything when `flex-wrap: wrap` is on.

CYCLE 3 · 27 MIN

THE CARD ROW THAT WRAPS BY ITSELF

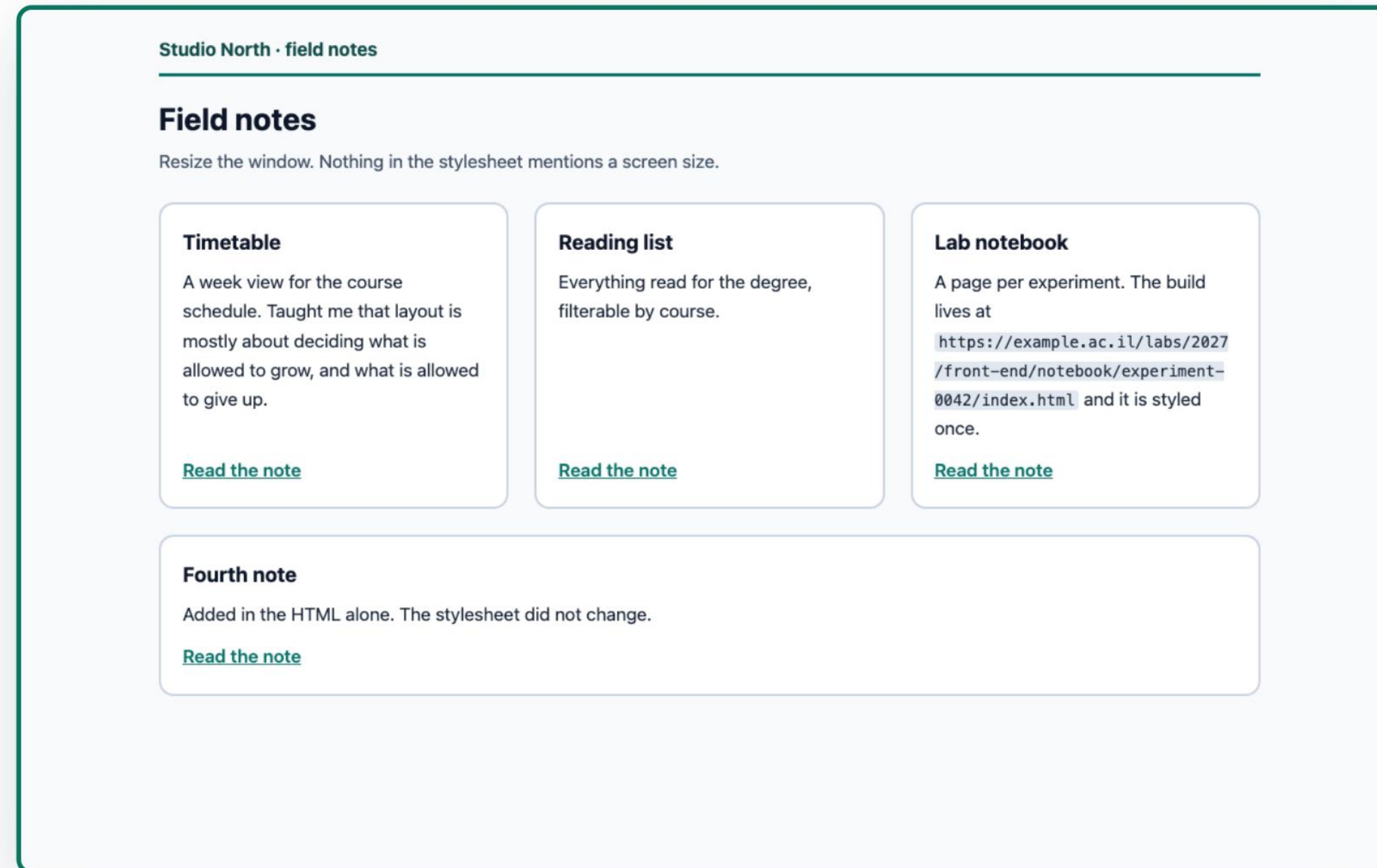
Three across when there is room, one under the other when there is not, nothing broken in between — and not one line that mentions a screen.

1 I DO11 MIN 4 on the slides above, 7 in the editor · live-code/03-card-row-wraps · width fails, flex-basis wins, min-width: 0, the link at the bottom	2 WE DO6 MIN type it · checkpoint 3 · a fourth card with no CSS change	3 YOU DO8 MIN a pricing row: three tiers, buttons on one line at the bottom (the rest in the assignment)	4 DEBRIEF2 MIN width vs flex-basis · min-width: auto · ↗ deeper
--	--	--	---

YOU DO · AI-FREE

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/03-card-row-wraps/checkpoint.png` is this picture at 1280; `live-code/03-card-row-wraps/end/` is the code behind it. Three across, equal height, links on one line, the URL broken inside its card, the fourth card alone and full width below.

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid.

BEFORE CYCLE 4 · WORDS YOU WILL USE

@media (min-width: 48em) — THE ONE YOU ARE ALLOWED

A media query asks the **viewport** a question: how wide is the visible part of the page? The rules inside the braces apply only while the answer is yes.

min-width **Mobile-first:** the phone is the default, the query adds the laptop.

48em, not 768px Relative to the browser's font size — enlarged text gets the simple layout sooner.

viewport The window you see the page through: the whole screen on a phone.

```
1  /* phone, and everything: */
2  .menu-toggle { display: inline-block; }
3
4  @media (min-width: 48em) {
5      /* laptop and wider only: */
6      .menu-toggle { display: none; }
7  }
```

Week 5 is media queries properly. This week: this one, no other.

THE WHOLE QUERY, LINE BY LINE

the toggle

Hidden above `48em`: there is room, so no button.

the two given lines

A closed `details` is zero width without its summary; its contents are skipped via `::details-content` (Baseline 2025).

the list, both states

Back to `static` and a row, whether the menu was open or closed on the phone.

```
1  @media (min-width: 48em) {
2    .menu-toggle { display: none; }
3
4    /* zero width without its summary */
5    .menu { flex: 1; }
6
7    /* painted whatever the open state */
8    .menu::details-content {
9      content-visibility: visible;
10   }
11
12   .menu-list,
13   .menu[open] .menu-list {
14     position: static;
15     flex-direction: row;
16   }
17 }
```

CYCLE 4 · BEFORE YOU PROMPT

TWO MINUTES ON `float`, BECAUSE YOU WILL MEET IT

demos/07-archaeology – five ways, 1998 to 2017↻ reloadopen ↗

1 · tables (1998)

2 · inline-block (2007)

3 · absolute (2005)

4 · float (2010)

5 · flex (2017)

THE SAME THREE COLUMNS. THE MARKUP NEVER CHANGES.

Column one, with a sentence in it so the boxes have something to be.

Column two.

Column three, which is the longest of the three by some way.

A paragraph after the columns.

```
.cols { display: table; width: 100%; border-spacing: 5px }  
.col { display: table-cell; width: 33.33% }
```

What it cost: the layout lives in the markup. A screen reader announces a

`float` was made to wrap text around an image; in 2010 it was bent into columns. A floated box leaves the flow, so its parent shrinks to no height — a **clearfix** is the old hack that makes the parent wrap its floats again.

► **HANDS-ON** `demos/07-archaeology/` — Click through in order and read the "what it cost" line. Then stop.

CYCLE 4 · 25 MIN

ONE PROMPT, TWICE: "PUT THESE THREE CARDS IN A ROW"

Paste a stylesheet that has one float in it and ask for a row. What comes back is 2010 — correct at 1280, wrong at the width you did not check. Then ask again with constraints.

1 I DO8 MIN 3 on the slides above, 5 in the chat · ai-lab/04-floats-or-flex · P1 live · run it at 1280, then at 375 · transcript.md if the room got something else	2 WE DO6 MIN everyone pastes P1 · find what is wrong before it is named	3 YOU DO9 MIN write P2 yourself, mobile-first, at most one query · run it at three widths, then drag 1600 to 320 · a journal line: claim · checked · verdict	4 DEBRIEF2 MIN the claim that has to be checked between X and Y · ↗ exemplars
--	---	--	---

YOU DO · AI AS TUTOR

CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you open the answer to P1 **at 375 and at 1280** and drag the width slowly from 1600 to 320 — not just stop at three widths?
2. Does your P2 contain at least two conditions a browser can check — "one column at 375", "at most one media query" — and does the answer meet both when you check?
3. Does your journal line contain a **width or a measurement** — a number, a tool, what you typed in the console — and not just "worked"?

Compare — `ai-lab/04-floats-or-flex/checkpoint-he.md` has the same three questions; `artifact/claude-p1/` and `artifact/claude-p2/` are what came back, ready to open.

WHAT `float` IS STILL FOR

Wrapping text around an image. That is what it was invented for in 1996, and it is still the only thing it does that nothing else does.

Today, using floats for overall page layout is discouraged.

If you find `float: left` on a column in a stylesheet you have inherited, you are looking at 2010. Replace it with a flex row and delete the clearfix.

```
1 .pull-quote {  
2   float: inline-start;  
3   width: 14rem;  
4   margin-inline-end: 1rem;  
5 }
```

Legitimate. Text flows around it, which is exactly the job.

CYCLE 5 · BEFORE YOU TYPE

RESPONSIVE, WITH NOTHING THAT MENTIONS A SCREEN — AND THE ONE THING THAT DOES

THE WHOLE CARD ROW

```
1 .card-row {
2   display: flex;
3   flex-wrap: wrap;
4   gap: 1.5rem;
5 }
6
7 .card {
8   flex: 1 1 17rem; /* I want 17rem.
9                     If you cannot,
10                      take a line. */
11 }
```

Three across at 1280. Two at 900. One at 375. **No media query, no breakpoint, no arithmetic that has to add up to 100 by hand.**

`flex-wrap` says items may take a new line rather than overflow; `flex-basis` says how much room each one wants before it gives up.

The one thing they cannot do: hide a Menu button on a laptop and show it on a phone. That is a question about the *viewport*, and it gets the one `@media (min-width: 48em)` in the whole file.

CYCLE 5 · 20 MIN

THE MENU THAT CLOSES ON A PHONE

Everything else on the page adapts with no media query. This is the one question Flexbox cannot answer — so this is the one query you are allowed.

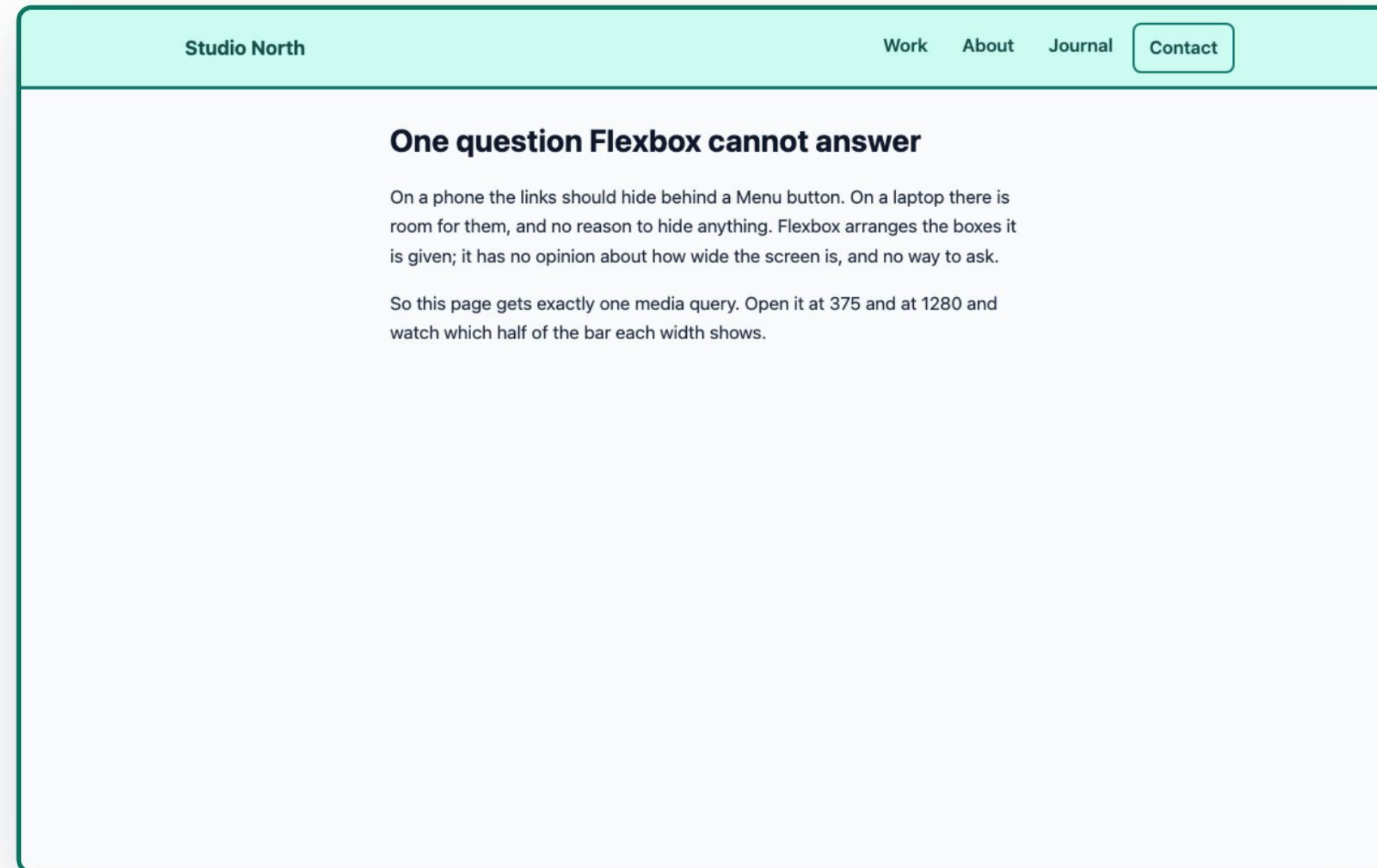
1 I DO7 MIN 2 on the slides above, 5 in the editor (wrapping the block in the query is copy this) · live-code/05-menu-closes-on-mobile · the rules outside the query, then inside · the two given lines · the overlay	2 WE DO6 MIN type it · checkpoint 5 · check both widths	3 YOU DO5 MIN Contact first in the open panel with order · dividers · both undone above 48em	4 DEBRIEF2 MIN how many queries are in your file? · ↗ deeper
--	---	--	--

YOU DO · AI AS TUTOR



CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/05-menu-closes-on-mobile/checkpoint.png` is this picture at 1280; `live-code/05-menu-closes-on-mobile/end/` is the code behind it. At 1280: links in a row, no button. Now 375 yourself: a Menu button, and the panel floats over the heading when opened — the heading does not move.



ASSIGNMENT · 145–175

THE ASSIGNMENT: ASSEMBLE WHAT YOU BUILT, THEN EXTEND IT

ASSEMBLE · CORE

`week-04/` is already in your repo from `pre-class/`. The navbar is cycle 1 + cycle 5. The badge and the sticky header are cycle 2. The card row is cycle 3. The hero is cycle 3 with two items instead of three.

EXTEND · STRETCH AND CHALLENGE

The pricing row is cycle 3's "you do" on the assignment's markup. The badge, the sticky header and the footer row are one declaration each — you typed all of them today.

THE RULES, AND THE EXAM

Flexbox only. No Grid, no float, one media query. Al as tutor — it explains, you type. Submit the SHA in Moodle. The exam (week 13) asks you about this code, offline.



DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU LEAVE

1. A badge with `position: absolute; top: 0; right: 0` lands in the corner of the page instead of the corner of its card. What is missing, and on which element?
2. A dropdown has `z-index: 999`. A header has `z-index: 10`. The dropdown is behind. Give me one reason — and tell me why raising 999 will not help.
3. `flex-direction: column`. Which property now centres the items left-to-right, and which one distributes them top-to-bottom?

► **GO DEEPER** — every slide marked *Deeper · self-study* is optional depth and is not examined: every value of `position` in one table, the overflow values, every Flexbox container property and the `flex` shorthand in tables, the warning on `order`, the whole menu query line by line, and the demos behind today's cycles. Each one sits after the slide it extends.

THE FIVE THAT WILL COST YOU TIME

1 · ABSOLUTE WITH NO ANCHOR

It aligned to the page, not to your box. `position: relative` is missing on the **ancestor**.

2 · Z-INDEX ACROSS CONTEXTS

999 lost to 10. An ancestor has `opacity`, `transform` or `filter`. Raising the number cannot help.

3 · JUSTIFY / ALIGN SWAPPED

You changed `flex-direction` and the axes went with it. Neither one is "horizontal".

4 · MARGIN: AUTO EVERYWHERE

It absorbs free space — that is all. Legitimate for centring a block, and for pushing one flex item to the end.

5 · HEIGHT: 100% DOING NOTHING

100% of `auto` is `auto`. Use `min-height: 100dvh`.

AND THE BONUS · MIN-WIDTH

A flex item will not shrink below its content. `min-width: 0` is the fix, and it is on the item.

WHERE TO LOOK THINGS UP

REFERENCE

MDN — search "mdn flexbox", "mdn position". The *Formal definition* table on each page tells you the default and whether it inherits.

PRACTICE

Flexbox Froggy — twenty-four levels, twenty minutes, free. Do it once and the property names stop needing to be looked up.

IN THE BROWSER

DevTools marks every flex container with a `flex` badge. Click it and the axes, the gaps and the free space are drawn on the page.

The demos are yours: `demos/06-flex/playground/` answers most questions faster than a search does.